StockFlow MVP

API Guide & Documentation

StockFlow is a multi-tenant SaaS inventory management application built on Next.js 16 (App Router) and Prisma 7.

Ø<ßÛþ System Architecture & API Pattern

Rather than traditional REST endpoints (`/api/products`), StockFlow leverages **React Server Actions** as its secure API framework.

- **Mechanism:** All mutations are defined in [src/app/actions.js](file:///c:/Antigravity/Project_WexaAI/src/app/actions.js) and marked with `"use server"`. Behind the scenes, Next.js compiles these into secure HTTP `POST` endpoints with cryptographically signed tokens.
- **Security:** Every inventory management and setting action calls `getSession()` to verify the user's active session. Data queries are automatically scoped by `organizationId` to prevent cross-tenant data leaks.
- **State Syncing:** Success responses trigger `revalidatePath()`, purging Next.js layouts and instantly updating the client views.

Ø=Ý 1. Authentication API

A. Register Organization & User (`registerAction`)

Registers a new tenant organization and the primary admin account.

- **Type:** Server Action Form Handler
- **Parameters:** `(prevState, formData)`
- **Input Payload (`FormData`):**
  - `email` *(string, required)*: Valid email address.
  - `password` *(string, required)*: Minimum 6 characters.
  - `name` *(string, required)*: Full name of the user.
  - `orgName` *(string, required)*: Store or company name.
- **Logic Flow:**

1. Checks if email is already in use.
2. Generates a random cryptographic salt and hashes the password using PBKDF2 (SHA-512).
3. Executes a database transaction (`db.$transaction`) to create both the `Organization` and `User` atomically.
4. Encrypts the session using AES-256-GCM and sets the secure `stockflow_session` cookie.

- **Return Value:**

```json
{ "success": true } // or { "error": "Error message description" }
```

B. User Login (`loginAction`)

Authenticates an existing user and starts an encrypted session.

- **Type:** Server Action Form Handler
- **Parameters:** `(prevState, formData)`
- **Input Payload (`FormData`):**
  - `email` *(string, required)*

    - `password` *(string, required)*
  - **Logic Flow:**
1. Queries user record by email.
2. Verifies the password using the saved salt and hash.
3. Sets the encrypted cookie session.
  - **Return Value:**
```json
{ "success": true } // or { "error": "Invalid email or password." }
```

C. User Logout (`logoutAction`)

Destroys the active session cookie.

- **Type:** Server Action
- **Parameters:** None
- **Return Value:**

```json
{ "success": true }
```

Ø=Üæ 2. Product Catalog API

A. Create Product (`createProductAction`)

Appends a new SKU to the organization's catalog.

- **Type:** Server Action Form Handler
- **Parameters:** `(prevState, formData)`
- **Input Payload (`FormData`):**
  - `name` *(string, required)*
  - `sku` *(string, required)*: Automatically converted to uppercase.
  - `description` *(string, optional)*
  - `quantityOnHand` *(integer, optional)*: Default `0`.
  - `costPrice` *(number, optional)*
  - `sellingPrice` *(number, optional)*
  - `lowStockThreshold` *(integer, optional)*: If omitted, inherits organization default.
- **Validation Rules:**
  - SKU must be unique within the user's organization.
  - Pricing and quantities must be positive.
- **Logic Flow:**

1. Validates input types and checks SKU collisions in the database.
2. Creates the product and automatically writes an initial load record to `StockAdjustmentLog` if quantity > 0.

- **Return Value:**

```json
{ "success": true } // or { "error": "SKU already exists." }
```

B. Update Product (`updateProductAction`)

Edits the properties of an existing catalog item.

- **Type:** Server Action
- **Parameters:** `(productId, formData)`
- **Input Payload:**
  - `productId` *(string, path parameter)*: Target product UUID.
  - `formData` *(FormData)*: Updated properties (`name`, `sku`, `description`, `costPrice`, `sellingPrice`, `lowStockThreshold`, and optionally `quantityOnHand`).
- **Logic Flow:**

1. Verifies ownership of the product.
2. Validates uniqueness if the SKU was modified.
3. If `quantityOnHand` differs from the current database value, it calculates the difference and automatically registers a `StockAdjustmentLog` (e.g. "Direct quantity edit").

- **Return Value:**

```json
{ "success": true } // or { "error": "Error description" }
```

C. Delete Product (`deleteProductAction`)

Hard-deletes an item and cleans up dependencies.

- **Type:** Server Action
- **Parameters:** `(productId)`
- **Input Payload:**
  - `productId` *(string)*: Target product UUID.
- **Return Value:**

```json
{ "success": true } // or { "error": "Product not found." }
```

Ø=ÜÊ 3. Inventory & Settings API

A. Adjust Stock (`adjustStockAction`)

Increments or decrements inventory levels, creating audit records.

- **Type:** Server Action
- **Parameters:** `(productId, quantityChange, note)`
- **Input Payload:**
  - `productId` *(string)*: Target product UUID.
  - `quantityChange` *(integer)*: Positive or negative delta (e.g., `+10` or `-3`).
  - `note` *(string, optional)*: Reason (e.g., "damaged", "weekly restock").
- **Validation Rules:**
  - Resulting stock count cannot drop below zero.
- **Logic Flow:**

1. Fetches current product stock.
2. Validates new total.
3. Atomically updates the stock count and logs the event in `StockAdjustmentLog`.

- **Return Value:**

```json
{ "success": true } // or { "error": "Insufficient stock." }
```

```

### B. Update Settings (`updateSettingsAction`)

Configures fallback parameters for the tenant store.

- **Type:** Server Action
- **Parameters:** `(defaultThreshold)`
- **Input Payload:**
  - `defaultThreshold` *(integer)*: Global fallback threshold limit.
- **Return Value:**

```json
{ "success": true } // or { "error": "Threshold must be positive." }
```
```